LAPORAN PRAKTIKUM MODUL 2

Dasar Pemrograman JavaScript
Variabel • Tipe Data • Operator • Kondisional • Perulangan

DISUSUN OLEH :

SAQYA RAHMATILLAH
2025573010131

Kelas : TI 1C
Mata Kuliah : Praktikum Algoritma dan Struktur Data
Jurusan : Teknologi Informasi dan Komputer
Prodi : Teknik Informatika
Dosen : Muhammad Reza Zulman, S.ST.,M.Sc

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

POLITEKNIK NEGERI LHOKSEUMAWE

2026

KATA PENGANTAR

Puji syukur saya panjatkan kehadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya saya dapat menyelesaikan laporan praktikum yang berjudul *"Dasar Pemrograman JavaScript (Modul 2)"* dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas pada mata kuliah Praktikum Algoritma dan Struktur Data. Dalam praktikum ini, saya mempelajari dasar-dasar pemrograman JavaScript, mulai dari inisiasi project menggunakan Node.js dan npm, penggunaan variabel dan tipe data, operator, hingga penerapan kondisional dan perulangan dalam pembuatan program sederhana.

Saya menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, saya sangat mengharapkan kritik dan saran yang membangun dari pembaca agar laporan ini dapat menjadi lebih baik di masa yang akan datang.

Akhir kata, saya berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca, khususnya dalam memahami dasar-dasar pemrograman JavaScript.

BAB I

PENDAHULUAN

1.1 Latar Belakang

JavaScript merupakan salah satu bahasa pemrograman yang sangat penting dalam dunia teknologi, khususnya dalam pengembangan website. JavaScript digunakan untuk membuat halaman web menjadi lebih interaktif, misalnya menampilkan data secara dinamis, memberikan respon terhadap aksi pengguna, dan mengatur logika dalam aplikasi web.

Pada saat ini, JavaScript tidak hanya digunakan di browser, tetapi juga dapat dijalankan di luar browser menggunakan Node.js. Hal ini membuat JavaScript menjadi bahasa yang sangat fleksibel dan banyak digunakan dalam berbagai bidang, baik front-end maupun back-end.

Oleh karena itu, mahasiswa perlu memahami dasar-dasar JavaScript terlebih dahulu sebelum mempelajari materi yang lebih lanjut. Praktikum ini dilakukan agar mahasiswa dapat belajar langsung melalui praktik, sehingga lebih mudah memahami konsep yang ada.

1.2 Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

- Melakukan inisiasi project javascript dengan npm.
- Memahami perbedaan var, let, dan const serta memilih yang tepat
- Mengenali dan menggunakan tipe data dasar: string, number, boolean, null, undefined
- Menggunakan operator aritmatika, perbandingan, dan logika
- Membuat percabangan logika dengan if/else dan switch
- Membuat perulangan dengan for dan while
- Melakukan commit dan push hasil kerja ke GitHub

BAB II

DASAR TEORI

2.1 npm dan package.json

npm (Node Package Manager) adalah manajer paket bawaan Node.js yang digunakan untuk membuat project, menginstal library, dan menjalankan skrip. Setiap project Node.js memiliki sebuah file bernama package.json yang berfungsi sebagai "kartu identitas" project.

File package.json menyimpan informasi seperti nama project, versi, dan daftar library yang digunakan. Meskipun di praktikum ini kita belum menginstal library apapun, membiasakan dirimenginisialisasi npm init sejak awal adalah kebiasaan yang baik dalam pengembangan perangkat lunak.

2.2 Variabel.

Variabel adalah tempat untuk menyimpan data dalam program. Dalam JavaScript, terdapat tiga cara untuk mendeklarasikan variabel, yaitu let, const, dan var, yang masing-masing memiliki karakteristik berbeda. Variabel yang dideklarasikan menggunakan let bersifat *mutable*, artinya nilainya dapat diubah setelah dideklarasikan. Oleh karena itu, let biasanya digunakan untuk variabel yang nilainya dapat berubah selama program berjalan.

Berbeda dengan let, const digunakan untuk menyimpan nilai yang bersifat tetap atau konstan. Setelah dideklarasikan, nilai pada variabel const tidak dapat diubah, sehingga cocok digunakan ketika nilai tersebut tidak perlu mengalami perubahan.

Sementara itu, var merupakan cara lama dalam mendeklarasikan variabel di JavaScript. Variabel dengan var memiliki perilaku *function scope* yang sering kali kurang terprediksi dalam penggunaannya. Karena itu, dalam penulisan kode modern, penggunaan var sudah tidak disarankan dan lebih baik digantikan dengan let atau const.

2.3 Tipe Data

Setiap nilai yang disimpan dalam variabel memiliki tipe data. Tipe data menentukan jenis nilai apa yang bisa disimpan dan operasi apa yang bisa dilakukan terhadapnya.

JavaScript memiliki 5 tipe data primitif yang perlu dikuasai:

- string — Teks yang diapit tanda kutip. Contoh: 'Halo', "Dunia", ` Template
- number — Angka, baik bulat maupun desimal. Contoh: 42, 3.14, -7
- boolean — Hanya dua nilai: true atau false. Digunakan untuk logika.
- null — Mewakili "tidak ada nilai" secara disengaja. Programmer yang menetapkan ini.
- undefined — Variabel sudah dideklarasikan tapi belum diberi nilai sama sekali.

Gunakan operator typeof untuk mengecek tipe data sebuah variabel saat runtime.

2.4 Operator

Operator adalah simbol yang digunakan untuk melakukan operasi pada satu atau lebih nilai (operand). Ada tiga kelompok operator yang paling sering digunakan:

- Operator Aritmatika — untuk perhitungan matematika: + - * / % **. Operator % (modulo) menghasilkan sisa bagi. Contoh: 10 % 3 = 1 (karena 10 = 3×3 + sisa 1).
- Operator Perbandingan — membandingkan dua nilai dan menghasilkan boolean true/false. Selalu gunakan === (strict equality) bukan ==. Operator === membandingkan nilai DAN tipe data, sehingga 5 === '5' menghasilkan false.
- Operator Logika — menggabungkan atau membalik kondisi boolean. && (AND): true jika keduanya true. || (OR): true jika salah satu true. ! (NOT): membalik nilai boolean.

2.5 Kondisional

Dalam kehidupan nyata, kita selalu membuat keputusan berdasarkan kondisi. Misalnya: *"Jika hujan, bawa payung. Jika tidak, tidak perlu."* Di pemrograman, logika seperti ini disebut struktur kondisional atau percabangan.

JavaScript menyediakan beberapa cara untuk menulis kondisional:

- if / else if / else — Paling fleksibel. Cocok untuk kondisi yang kompleks atau melibatkan rentang nilai (misalnya menentukan grade dari nilai angka).
- switch — Cocok untuk mengecek satu variabel terhadap beberapa kemungkinan nilai yang spesifik (misalnya mencocokkan nama hari).
- Ternary Operator (kondisi ? nilaiJikaTrue : nilaiJikaFalse) — Versi singkat dari if/else yang menghasilkan sebuah nilai. Cocok untuk kondisi sederhana yang bisa ditulis dalam satu baris.

Konsep truthy dan falsy: Di JavaScript, nilai seperti 0, '' (string kosong), null, undefined, dan false dianggap sebagai falsy (bertindak seperti false). Semua nilai lainnya dianggap truthy.

2.6 Perulangan (Loop)

Bayangkan kamu ingin mencetak angka 1 sampai 1000. Menulis console.log(1) sampai console.log(1000) secara manual tentu tidak masuk akal. Perulangan (loop) adalah solusinya — sebuah blok kode yang dijalankan berulang kali selama suatu kondisi terpenuhi.

JavaScript memiliki beberapa jenis loop:

- for loop — Digunakan ketika kamu sudah tahu berapa kali perulangan akan berjalan. Terdiri dari tiga bagian: inisialisasi, kondisi, dan update.
- while loop — Digunakan ketika kamu belum tahu berapa kali perulangan berjalan, tapi tahu kapan harus berhenti. Loop berjalan selama kondisi bernilai true.

Dua kata kunci penting untuk mengontrol alur perulangan:

- break — Menghentikan perulangan sepenuhnya dan keluar dari loop.
- continue — Melewatkan sisa kode di iterasi saat ini dan langsung lanjut ke iterasi berikutnya.

Hati-hati dengan infinite loop — loop yang tidak pernah berhenti karena kondisinya tidak pernah menjadi false. Ini akan membuat program hang. Jika terjadi, tekan Ctrl+C untuk menghentikannya.

BAB III

LANGKAH-LANGKAH PRAKTIKUM

3.1 Persiapan Project

1. Buka Git Bash. Kemudian navigasikan ke folder repository GitHub kamu yang sudah di-clone.

2. Buat folder baru bernama praktikum-2.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData (mai
n)
$ mkdir praktikum-2

USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData (mai
n)
$ cd praktikum-2
```

3. Inisialisasi npm project. Perintah ini akan membuat file package.json secara otomatis.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/prak
tikum-2 (main)
$ npm init -y
```

3.2 Langkah Pengerjaan

3.2.1 Variabel

1. Di dalam folder praktikum-2/, buat file baru bernama 01-variabel.js.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData (main)
$ touch 01-variabel.js
```

2. Buka file 01-variabel.js di VS Code, lalu ketikkan kode berikut.

```js
// 01-variabel.js
// ==============================================
// VARIABEL DAN DEKLARASI
// ==============================================

// --- Deklarasi dengan let (nilai bisa diubah) ---
let nama = "Budi";
let umur = 20;
let kota = "Banda Aceh";

console.log("=== Data Mahasiswa ===");
console.log("Nama :", nama);
console.log("Umur :", umur);
console.log("Kota :", kota);

// --- Mengubah nilai let ---
umur = 21;
console.log("Ulang tahun! Umur sekarang:", umur);

// --- Deklarasi dengan const (nilai TIDAK bisa diubah) ---
const jurusan = "Teknik Informatika";
const tahunMasuk = 2023;

console.log("Jurusan    :", jurusan);
console.log("Tahun Masuk:", tahunMasuk);

// --- Coba hapus '//' di baris bawah ini, lalu jalankan ulang ---
// jurusan = 'Sistem Informasi';  // --> Ini akan menyebabkan ERROR!
```

3. Simpan file (Ctrl+S atau Cmd+S), kemudian jalankan file tersebut dari terminal.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-2 (main)
$ node 01-variabel.js
=== Data Mahasiswa ===
Nama : Budi
Umur : 20
Kota : Banda Aceh
Ulang tahun! Umur sekarang: 21
Jurusan     : Teknik Informatika
Tahun Masuk: 2023
```

3.2.2 Tipe Data

1. Buat file baru bernama 02-tipe-data.js di dalam folder praktikum-2/.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-2 (ma
in)
$ touch 02-tipe-data.js
```

2. Buka file 02-tipe-data.js di VS Code, lalu ketikkan kode berikut.

```javascript
// 02-tipe-data.js
// ==============================================
// MENGENAL TIPE DATA JAVASCRIPT
// ==============================================

// --- 1. STRING (teks) ---
let namaMahasiswa = "Ahmad Fauzi";
let programStudi = "Teknik Informatika";

// Template literal: gunakan backtick (`) untuk menggabungkan teks & variabel
let perkenalan = `Halo! Nama saya ${namaMahasiswa} dari ${programStudi}.`;
console.log(perkenalan);
console.log("Panjang nama:", namaMahasiswa.length); // property .length

// --- 2. NUMBER (angka) ---
let nilaiUjian = 87; // bilangan bulat
let ipk = 3.75; // bilangan desimal
let suhuKulkas = -4; // bilangan negatif

console.log("Nilai Ujian :", nilaiUjian);
console.log("IPK        :", ipk);
console.log("Suhu Kulkas:", suhuKulkas);

// --- 3. BOOLEAN (true / false) ---
let sudahLogin = true;
let sudahLulus = false;

console.log("Sudah login :", sudahLogin);
console.log("Sudah lulus :", sudahLulus);

// --- 4. NULL (nilai kosong yang disengaja) ---
let fotoProfil = null; // belum ada foto
console.log("Foto profil:", fotoProfil);

// --- 5. UNDEFINED (belum diberi nilai) ---
let nomorTelepon;
console.log("No. Telepon:", nomorTelepon);

// --- Mengecek tipe data dengan typeof ---
console.log("--- Tipe Data ---");
console.log("namaMahasiswa :", typeof namaMahasiswa); // string
console.log("nilaiUjian    :", typeof nilaiUjian); // number
console.log("sudahLogin    :", typeof sudahLogin); // boolean
console.log("fotoProfil    :", typeof fotoProfil); // object <- keanehan JS!
console.log("nomorTelepon  :", typeof nomorTelepon); // undefined
```

3. Simpan file, kemudian jalankan dari terminal.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-2 (main)
$ node 02-tipe-data.js
Halo! Nama saya Ahmad Fauzi dari Teknik Informatika.
Panjang nama: 11
Nilai Ujian : 87
IPK         : 3.75
Suhu Kulkas: -4
Sudah login : true
Sudah lulus : false
Foto profil: null
No. Telepon: undefined
--- Tipe Data ---
namaMahasiswa : string
nilaiUjian    : number
sudahLogin    : boolean
fotoProfil    : object
nomorTelepon  : undefined
```

3.2.3 Operator

1. Buat file baru bernama 03-operator.js di dalam folder praktikum-2/.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-2 (ma
in)
$ touch 03-operator.js
```

2. Buka file 03-operator.js di VS Code, lalu ketikkan kode berikut.

```javascript
// 03-operator.js
// ================================================
// OPERATOR JAVASCRIPT
// ================================================

// --- OPERATOR ARITMATIKA ---
let a = 17;
let b = 5;

console.log("=== Aritmatika ===");
console.log(`${a} + ${b} = ${a + b}`); // 22
console.log(`${a} - ${b} = ${a - b}`); // 12
console.log(`${a} * ${b} = ${a * b}`); // 85
console.log(`${a} / ${b} = ${a / b}`); // 3.4
console.log(`${a} % ${b} = ${a % b}`); // 2  (sisa bagi: 17 = 5x3 + 2)
console.log(`${a} ** ${b} = ${a ** b}`); // 1419857 (17 pangkat 5)

// --- OPERATOR PERBANDINGAN ---
console.log("=== Perbandingan ===");
console.log("10 > 5   :", 10 > 5); // true
console.log("10 < 5   :", 10 < 5); // false
console.log("10 >= 10 :", 10 >= 10); // true
console.log("10 <= 9  :", 10 <= 9); // false
```

```
24
25  // Perbedaan == dan ===
26  console.log("--- Perbedaan == dan === ---");
27  console.log('5 == "5"  :', 5 == "5"); // true  (JANGAN gunakan ini!)
28  console.log('5 === "5" :', 5 === "5"); // false (SELALU gunakan ini)
29  console.log("5 !== 3  :", 5 !== 3); // true  (tidak sama dengan)
30
31  // --- OPERATOR LOGIKA ---
32  console.log("=== Logika ===");
33  let sudahMakan = true;
34  let punyaUang = false;
35
36  // AND (&&): kedua kondisi harus true
37  console.log("Makan && Uang  :", sudahMakan && punyaUang); // false
38
39  // OR (||): salah satu kondisi cukup true
40  console.log("Makan || Uang  :", sudahMakan || punyaUang); // true
41
42  // NOT (!): membalik nilai boolean
43  console.log("!sudahMakan    :", !sudahMakan); // false
44  console.log("!punyaUang     :", !punyaUang); // true
```

3. Simpan file, kemudian jalankan dari terminal.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-2 (main)
$ node 03-operator.js
=== Aritmatika ===
17 + 5 = 22
17 - 5 = 12
17 * 5 = 85
17 / 5 = 3.4
17 % 5 = 2
17 ** 5 = 1419857
=== Perbandingan ===
10 > 5   : true
10 < 5   : false
10 >= 10 : true
10 <= 9  : false
--- Perbedaan == dan === ---
5 == "5"  : true
5 === "5" : false
5 !== 3  : true
=== Logika ===
Makan && Uang  : false
Makan || Uang  : true
!sudahMakan    : false
!punyaUang     : true
```

4. Kerjakan Latihan 1: Kalkulator Sederhana. Tambahkan kode baru di bawah kode yang sudah ada di file 03-operator.js (jangan hapus kode sebelumnya).

```javascript
// Deklarasi variabel
const panjang = 12;
const lebar = 8;

// Perhitungan
const luas = panjang * lebar;
const keliling = 2 * (panjang + lebar);

// Output menggunakan template literal
console.log(`Luas persegi panjang: ${luas}`);
console.log(`Keliling persegi panjang: ${keliling}`);

// Pengecekan apakah luas > 100
const lebihDari100 = luas > 100;
console.log(`Apakah luas lebih besar dari 100? ${lebihDari100}`);
```

5. Simpan dan jalankan ulang dengan node 03-operator.js. Pastikan tidak ada error.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-2 (main)
$ node 03-operator.js
=== Aritmatika ===
17 + 5 = 22
17 - 5 = 12
17 * 5 = 85
17 / 5 = 3.4
17 % 5 = 2
17 ** 5 = 1419857
=== Perbandingan ===
10 > 5   : true
10 < 5   : false
10 >= 10 : true
10 <= 9  : false
--- Perbedaan == dan === ---
5 == "5"  : true
5 === "5" : false
5 !== 3  : true
=== Logika ===
Makan && Uang  : false
Makan || Uang  : true
!sudahMakan    : false
!punyaUang     : true
Luas persegi panjang: 96
Keliling persegi panjang: 40
Apakah luas lebih besar dari 100? false
```

3.2.4 Kondisional

1. Buat file baru bernama 04-kondisional.js di dalam folder praktikum-2/.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-2 (ma
in)
$ touch 04-kondisional.js
```

2. Buka file 04-kondisional.js di VS Code, lalu ketikkan kode berikut.

```javascript
// 04-kondisional.js
// ================================================
// KONDISIONAL: if/else, switch, ternary
// ================================================

// --- IF / ELSE IF / ELSE: Konversi nilai ke grade ---
let nilaiUjian = 78;

console.log("=== Konversi Grade ===");
console.log("Nilai:", nilaiUjian);

if (nilaiUjian >= 90) {
  console.log("Grade: A (Sangat Baik)");
} else if (nilaiUjian >= 80) {
  console.log("Grade: B (Baik)");
} else if (nilaiUjian >= 70) {
  console.log("Grade: C (Cukup)");
} else if (nilaiUjian >= 60) {
  console.log("Grade: D (Kurang)");
} else {
  console.log("Grade: E (Tidak Lulus)");
}

// --- SWITCH: Cek nama hari ---
let namaHari = "Rabu";

console.log("\n=== Cek Jenis Hari ===");
switch (namaHari) {
  case "Senin":
  case "Selasa":
  case "Rabu":
  case "Kamis":
  case "Jumat":
    console.log(`${namaHari} adalah hari kerja.`);
    break;
  case "Sabtu":
  case "Minggu":
    console.log(`${namaHari} adalah akhir pekan.`);
    break;
  default:
    console.log("Nama hari tidak dikenali.");
}

// --- TERNARY OPERATOR: Cek kelulusan ---
let nilaiAkhir = 65;
let statusLulus = nilaiAkhir >= 60 ? "LULUS" : "TIDAK LULUS";

console.log("\n=== Status Kelulusan ===");
console.log(`Nilai ${nilaiAkhir}: ${statusLulus}`);
```

3. Simpan file, kemudian jalankan dari terminal.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-2 (main)
$ node 04-kondisional.js
=== Konversi Grade ===
Nilai: 78
Grade: C (Cukup)

=== Cek Jenis Hari ===
Rabu adalah hari kerja.

=== Status Kelulusan ===
Nilai 65: LULUS
```

4. Kerjakan Latihan 2: Konverter Musim dan Status Cuaca. Tambahkan kode baru di bawah kode yang sudah ada di file 04-kondisional.js (jangan hapus kode sebelumnya).

```javascript
// Deklarasi variabel bulan
const bulan = 7;

// Menentukan musim
let musim;

if (bulan === 12 || bulan === 1 || bulan === 2) {
  musim = "Dingin";
} else if (bulan >= 3 && bulan <= 5) {
  musim = "Semi";
} else if (bulan >= 6 && bulan <= 8) {
  musim = "Panas";
} else if (bulan >= 9 && bulan <= 11) {
  musim = "Gugur";
} else {
  musim = "Bulan tidak valid";
}

// Output musim
console.log(`Bulan ${bulan} termasuk musim: ${musim}`);

// Deklarasi variabel boolean
const adaAwan = true;
const adaAngin = false;

// Operasi logika
const mendungBerangin = adaAwan && adaAngin;
const adaAwanAtauAngin = adaAwan || adaAngin;
const langitCerah = !adaAwan;

// Output hasil
console.log(`Apakah cuaca mendung sekaligus berangin? ${mendungBerangin}`);
console.log(`Apakah ada awan atau angin? ${adaAwanAtauAngin}`);
console.log(`Apakah langit cerah (tidak ada awan)? ${langitCerah}`);
```

5. Simpan dan jalankan ulang. Pastikan tidak ada error.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-2 (main)
$ node 04-kondisional.js
=== Konversi Grade ===
Nilai: 78
Grade: C (Cukup)

=== Cek Jenis Hari ===
Rabu adalah hari kerja.

=== Status Kelulusan ===
Nilai 65: LULUS
Bulan 7 termasuk musim: Panas
Apakah cuaca mendung sekaligus berangin? false
Apakah ada awan atau angin? true
Apakah langit cerah (tidak ada awan)? false
```

3.2.5 Perulangan (Loop)

1. Buat file baru bernama 05-perulangan.js di dalam folder praktikum-2/.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-2 (ma
in)
$ touch 05-perulangan.js
```

2. Buka file 05-perulangan.js di VS Code, lalu ketikkan kode berikut.

```javascript
// 05-perulangan.js
// ================================================
// PERULANGAN: for, while, break, continue
// ================================================

// --- FOR LOOP ---
// Struktur: for (inisialisasi; kondisi; update)
console.log("=== For Loop: Hitung 1 sampai 5===");
for (let i = 1; i <= 5; i++) {
  console.log(`Iterasi ke-${i}`);
}

// --- WHILE LOOP ---
console.log("\n=== While Loop: Countdown ===");
let hitung = 5;
while (hitung > 0) {
  console.log(`Hitung mundur: ${hitung}`);
  hitung--; // WAJIB: kurangi nilai agar loop tidak berjalan selamanya
}
console.log("Selesai!");

// --- FOR LOOP: Mencetak bilangan genap ---
console.log("\n=== Bilangan Genap antara 1 - 20 ===");
for (let i = 1; i <= 20; i++) {
  if (i % 2 === 0) {
    // jika i habis dibagi 2 (sisa = 0), maka genap
    process.stdout.write(i + " "); // cetak di baris yang sama
  }
}
console.log(""); // pindah baris
```

```
// --- BREAK dan CONTINUE ---
console.log("\n=== Break dan Continue ===");
for (let i = 1; i <= 10; i++) {
  if (i === 4) {
    console.log(`Melewati angka ${i} (continue)`);
    continue; // lewati angka 4, lanjut ke i=5
  }
  if (i === 8) {
    console.log(`Berhenti di angka ${i} (break)`);
    break; // hentikan loop di angka 8
  }
  console.log(`Angka: ${i}`);
}
```

3. Simpan file, kemudian jalankan dari terminal.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-2 (main)
$ node 05-perulangan.js
=== For Loop: Hitung 1 sampai 5===
Iterasi ke-1
Iterasi ke-2
Iterasi ke-3
Iterasi ke-4
Iterasi ke-5

=== While Loop: Countdown ===
Hitung mundur: 5
Hitung mundur: 4
Hitung mundur: 3
Hitung mundur: 2
Hitung mundur: 1
Selesai!

=== Bilangan Genap antara 1 - 20 ===
2 4 6 8 10 12 14 16 18 20

=== Break dan Continue ===
Angka: 1
Angka: 2
Angka: 3
Melewati angka 4 (continue)
Angka: 5
Angka: 6
Angka: 7
Berhenti di angka 8 (break)
```

4. Kerjakan Latihan 3: Segitiga Bintang dan Deret Fibonacci. Tambahkan kode baru di bawah kode yang sudah ada di file 05-perulangan.js.

```javascript
// BAGIAN A: Segitiga Bintang
console.log("Segitiga Bintang:");
for (let i = 1; i <= 5; i++) {
  let baris = "";
  for (let j = 1; j <= i; j++) {
    baris += "*";
  }
  console.log(baris);
}

// BAGIAN B: Deret Bilangan Prima 1 - 50
console.log("\nBilangan Prima (1-50):");

for (let i = 2; i <= 50; i++) {
  let isPrima = true;

  for (let j = 2; j < i; j++) {
    if (i % j === 0) {
      isPrima = false;
      break;
    }
  }

  if (isPrima) {
    console.log(i);
  }
}
```

5. Simpan dan jalankan ulang. Pastikan tidak ada error.

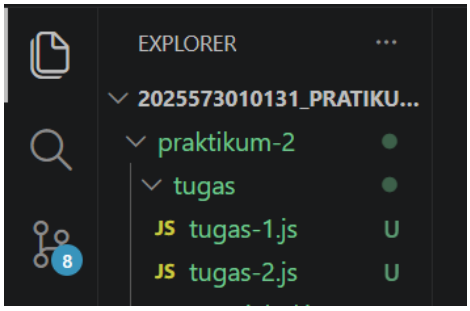
```
Segitiga Bintang:
*
**
***
****
*****

Bilangan Prima (1-50):
2
3
5
7
11
13
17
19
23
29
31
37
41
43
47
```

3.3 Langkah Pengerjaan Tugas Pratikum

1. Buat folder tugas/ dan dua file tugas di dalamnya, dengan menggunakan perintah:

mkdir tugas

touch tugas/tugas-1.js tugas/tugas-2.js



2. Ketik perintah berikut untuk masuk kedalaam folder tugas yang telah di buat

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-2 (main)
$ cd tugas
```

3.3.1 Tugas 1: FizzBuzz

1. Buka file tugas/tugas-1.js di VS Code.

2. Selanjutnya, dilakukan penulisan kode program untuk menjawab permasalahan yang diberikan pada tugas.

```javascript
for (let i = 1; i <= 50; i++) {
  if (i % 15 === 0) {
    console.log("FizzBuzz");
  } else if (i % 3 === 0) {
    console.log("Fizz");
  } else if (i % 5 === 0) {
    console.log("Buzz");
  } else {
    console.log(i);
  }
}
```

3. Simpan file, kemudian jalankan dari terminal.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-2/tugas (main)
$ node tugas-1.js
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
Fizz
22
23
Fizz
Buzz
26
Fizz
28
29
FizzBuzz
31
32
Fizz
```

```
34
Buzz
Fizz
37
38
Fizz
Buzz
41
Fizz
43
44
FizzBuzz
46
47
Fizz
49
Buzz
```

3.3.2 Tugas 2: Kalkulator BMI (Body Mass Index)

1. Buka file tugas/tugas-2.js di VS Code.

2. Selanjutnya, dilakukan penulisan kode program untuk menjawab permasalahan yang diberikan pada tugas.

```javascript
// 1. Deklarasi variabel
const beratBadan = 68; // dalam kg
const tinggiBadan = 1.72; // dalam meter

// 2. Hitung BMI
let bmi = beratBadan / (tinggiBadan * tinggiBadan);

// 3. Tampilkan BMI dengan 2 angka desimal
let bmiFixed = bmi.toFixed(2);

// 4. Tentukan kategori BMI
let kategori;

if (bmi < 18.5) {
  kategori = "Kurus (Underweight)";
} else if (bmi >= 18.5 && bmi <= 24.9) {
  kategori = "Normal (Ideal)";
} else if (bmi >= 25.0 && bmi <= 29.9) {
  kategori = "Kelebihan Berat Badan (Overweight)";
} else {
  kategori = "Obesitas (Obese)";
}

// 5. Tampilkan hasil akhir
console.log(
  `Berat: ${beratBadan} kg | Tinggi: ${tinggiBadan} m | BMI: ${bmiFixed} | Kategori: ${kategori}`,
);
```

3. Simpan file, kemudian jalankan dari terminal.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-2/tugas (main)
$ node tugas-2.js
Berat: 68 kg | Tinggi: 1.72 m | BMI: 22.99 | Kategori: Normal (Ideal)
```

3.4 Submit ke GitHub

1. Pastikan kamu berada di root folder repository (bukan di dalam praktikum-2/), lalu cek status file.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData (main)
$ cd praktikum-2
```

2. Tambahkan seluruh folder praktikum-2/ ke staging area.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-2 (main)
$ git add .
warning: in the working copy of 'praktikum-2/package.json', LF will be replaced by CRLF the next time Git
touches it
```

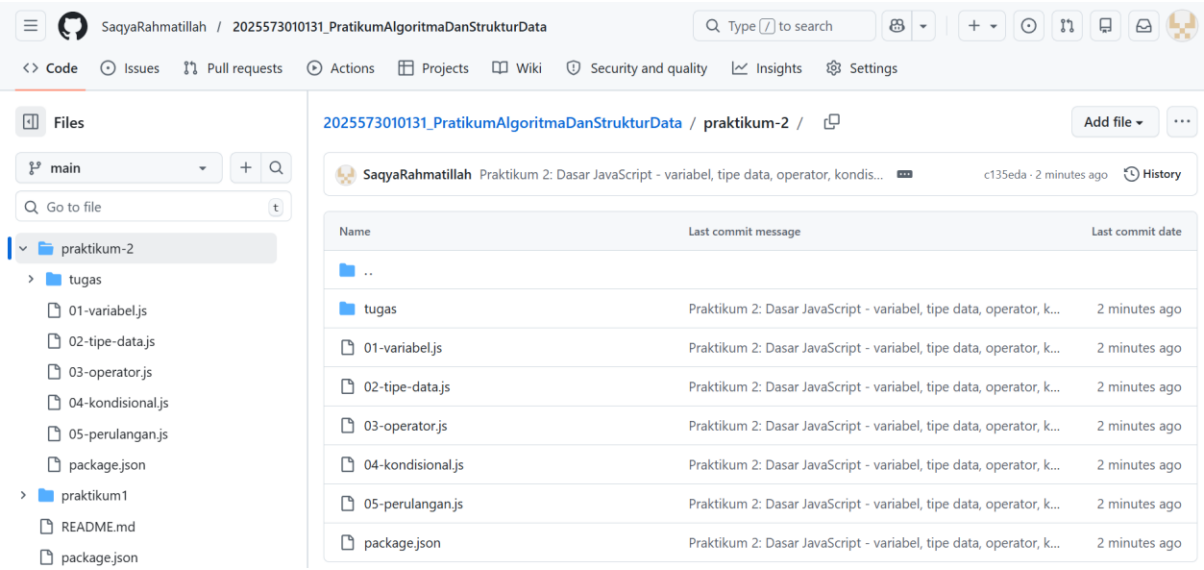
3. Buat commit dengan pesan yang deskriptif.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-2 (main)
$ git commit -m "Praktikum 2: Dasar JavaScript - variabel, tipe data, operator, kondisional,
perulangan"
[main c135eda] Praktikum 2: Dasar JavaScript - variabel, tipe data, operator, kondisional, perulangan
 8 files changed, 321 insertions(+)
 create mode 100644 praktikum-2/01-variabel.js
 create mode 100644 praktikum-2/02-tipe-data.js
 create mode 100644 praktikum-2/03-operator.js
 create mode 100644 praktikum-2/04-kondisional.js
 create mode 100644 praktikum-2/05-perulangan.js
 create mode 100644 praktikum-2/package.json
 create mode 100644 praktikum-2/tugas/tugas-1.js
 create mode 100644 praktikum-2/tugas/tugas-2.js
```

4. Push ke repository GitHub.

```
USER@DESKTOP-594SE9L MINGW64 ~/2025573010131_PratikumAlgoritmaDanStrukturData/praktikum-2 (main)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 12 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 4.45 KiB | 569.00 KiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/SaqyaRahmatillah/2025573010131_PratikumAlgoritmaDanStrukturData.git
   f27a19f..c135eda  main -> main
```

5. Project berhasil di unggah (push).



BAB IV

HASIL DAN PEMBAHASAN

Berdasarkan praktikum yang telah dilakukan, seluruh materi yang dipelajari pada modul ini dapat diimplementasikan dengan baik menggunakan JavaScript melalui Node.js. Proses dimulai dengan melakukan inisiasi project menggunakan npm yang menghasilkan file *package.json* sebagai identitas project. Hal ini menunjukkan bahwa mahasiswa sudah memahami tahap awal dalam pembuatan project JavaScript.

Pada bagian variabel, mahasiswa telah mencoba menggunakan let, const, dan var dalam penulisan program. Dari hasil percobaan, dapat dipahami bahwa penggunaan let lebih fleksibel karena nilainya dapat berubah, sedangkan const digunakan untuk nilai yang tetap. Sementara itu, penggunaan var mulai ditinggalkan karena memiliki perilaku yang kurang konsisten dibandingkan let dan const.

Selanjutnya pada bagian tipe data, program yang dibuat mampu menampilkan berbagai jenis data seperti string, number, dan boolean. Selain itu, penggunaan null dan undefined juga dapat dipahami sebagai kondisi ketika suatu variabel tidak memiliki nilai atau belum diisi. Dengan menggunakan operator typeof, tipe data dari suatu variabel dapat diketahui dengan mudah.

Pada bagian operator, mahasiswa telah menggunakan operator aritmatika untuk melakukan perhitungan, operator perbandingan untuk membandingkan nilai, serta operator logika untuk menggabungkan kondisi. Hasil yang ditampilkan sesuai dengan teori yang telah dipelajari, sehingga menunjukkan bahwa konsep operator sudah dipahami dengan baik.

Pada bagian kondisional, program berhasil menggunakan struktur if/else dan switch untuk menentukan keputusan berdasarkan kondisi tertentu. Contohnya seperti menentukan apakah suatu nilai termasuk lulus atau tidak. Hal ini menunjukkan bahwa mahasiswa sudah mampu memahami logika percabangan dalam pemrograman.

Selanjutnya pada bagian perulangan, penggunaan for dan while berhasil digunakan untuk menampilkan output secara berulang. Perulangan ini membantu dalam menyederhanakan kode sehingga tidak perlu menulis perintah yang sama secara berulang. Namun, perlu diperhatikan agar kondisi perulangan tidak menyebabkan *infinite loop*.

Secara keseluruhan, hasil praktikum menunjukkan bahwa semua konsep dasar JavaScript pada modul ini dapat dipahami dan diimplementasikan dengan baik. Praktikum ini sangat membantu dalam memperkuat pemahaman teori melalui praktik langsung.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa JavaScript merupakan bahasa pemrograman yang sangat penting dalam pengembangan web. Dengan mempelajari dasar-dasar seperti variabel, tipe data, operator, kondisional, dan perulangan, mahasiswa dapat memahami cara kerja logika dalam suatu program.

Selain itu, penggunaan Node.js dan npm juga memberikan pengalaman awal dalam membuat dan mengelola project JavaScript secara lebih terstruktur. Praktikum ini membantu mahasiswa untuk tidak hanya memahami teori, tetapi juga mampu mengimplementasikannya dalam bentuk program sederhana.

Dengan demikian, praktikum ini menjadi langkah awal yang penting dalam mempelajari pemrograman, khususnya JavaScript, sebelum masuk ke materi yang lebih kompleks.

5.2 Saran

Adapun saran yang dapat diberikan adalah mahasiswa sebaiknya lebih sering berlatih dalam menulis kode JavaScript agar semakin memahami konsep dasar yang telah dipelajari. Selain itu, mahasiswa juga disarankan untuk mencoba berbagai variasi program agar lebih terbiasa dalam menggunakan logika pemrograman.

Selanjutnya, mahasiswa dapat melanjutkan pembelajaran ke materi yang lebih lanjut seperti fungsi, array, dan objek, karena materi tersebut sangat penting dalam pengembangan program yang lebih kompleks.

Terakhir, dalam penulisan kode program, sebaiknya selalu memperhatikan kerapian dan struktur penulisan agar kode mudah dibaca, dipahami, dan dikembangkan di kemudian hari.